Наивный Байес и NLP

PIERIAN DATA

Наивный Байес и NLP

- В этом разделе курса мы посмотрим, как можно использовать сырые текстовые данные для моделей машинного обучения.
- Эта область машинного обучения называется “Natural Language Processing” (NLP) – обработка естественного языка.

Наивный Байес и NLP

- Обзор раздела
  - Наивный Байесовский алгоритм и NLP
  - Извлечение признаков из текстовых данных
  - Задача классификации текстов
- Обратите внимание:
  - Это будут задачи обучения с учителем (supervised learning). Задачи обучения без учителя (unsupervised learning) будут позже.

Давайте начнём!

Наивный Байес

Часть 1: Теорема Байеса

PIERIAN DATA

Обработка естественного языка (NLP)

- Наивный Байес – это целый набор алгоритмов, применяющих теорему Байеса для задач классификации. Это алгоритмы обучения с учителем (supervised learning).
- Теорема Байеса это формула вероятностей, вычисляющая вероятности связанных событий на основе уже известных вероятностей.

Обработка естественного языка (NLP)

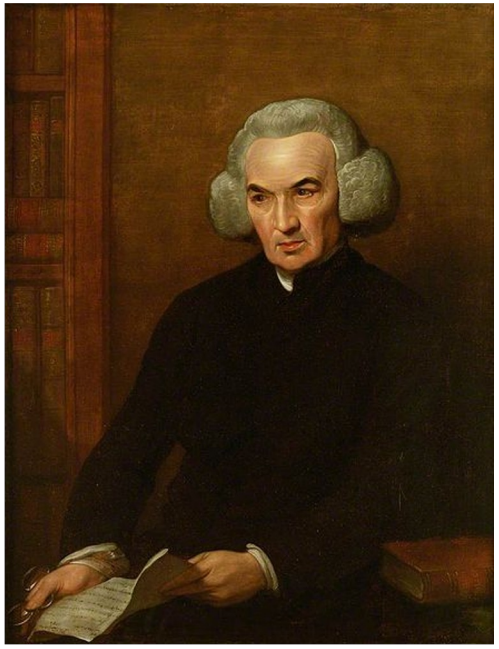
- 1700-е: Томас Байес был математиком и священником в Британии, занимался теологией, статистикой и логикой.

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

Обработка естественного языка (NLP)

- 1700-е: Теорема Байеса была опубликована уже после его жизни. Ричард Прайс отредактировал и опубликовал его записки.

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$



Обработка естественного языка (NLP)

- Теорема Байеса $$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$
  - A и B – это события.
  - P(A|B) – вероятность события A при условии, что событие B произошло.
  - P(B|A) – вероятность события B при условии, что событие A произошло.
  - P(A) – вероятность события A.
  - P(B) – вероятность события B.

Обработка естественного языка (NLP)

- Теорема Байеса
  - Давайте рассмотрим пример!

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

Обработка естественного языка (NLP)

- Представим следующую ситуацию в городе:
  - В каждой квартире в домах установлены датчики противопожарной сигнализации.
  - Однако, бывают ложные срабатывания – датчик выдаёт сигнал о наличии дыма, хотя настоящего пламени нету (например, только дым от пригоревшей еды на плите).

Обработка естественного языка (NLP)

- Вероятности различных событий:
  - Настоящие опасные возгорания случаются только в 1% случаев.
  - Датчики дыма некачественные, они срабатывают только в 10% случаев.
  - Когда действительно случилось возгорание, то датчик срабатывает в 95% случаев.

Обработка естественного языка (NLP)

- Мы ищем ответ на следующий вопрос:
  - Если сработал датчик дыма, то какова вероятность того, что это действительно опасное возгорание?

Обработка естественного языка (NLP)

- В терминах событий и вероятностей:
  - Событие A: Опасное возгорание
  - Событие B: Сработал датчик дыма
  - P(A|B): Вероятность возгорания, если сработал датчик дыма
  - P(B|A): Вероятность срабатывания датчика, если возгорание опасное

Обработка естественного языка (NLP)

- Вероятности различных событий:
  - Настоящие опасные возгорания случаются только в 1% случаев. **P(Fire) = 1/100**
  - Датчики дыма некачественные, они срабатывают в 10% случаев. **P(Smoke) = 1/10**
  - Когда действительно случилось возгорание, то датчик срабатывает в 95% случаев. **P(Smoke|Fire) = 95/100**

Обработка естественного языка (NLP)

- Теорема Байеса:

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

P(Fire|Smoke) = P(Smoke|Fire)*P(Fire)/P(Smoke)

Обработка естественного языка (NLP)

- Теорема Байеса:

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

P(Fire|Smoke) = P(Smoke|Fire)*P(Fire)/P(Smoke)
P(Fire|Smoke) = 0.95 * 0.01 / 0.1
P(Fire|Smoke) = 0.095
P(Fire|Smoke) = 9.5%

Обработка естественного языка (NLP)

- Далее мы посмотрим, как можно применить теорему Байеса для задач классификации данных.
- Мы будем применять эту теорему для задач классификации текстовых данных (natural language processing – обработка естественного языка).

Наивный Байес

Часть 2: Наивный Байес

PIERIAN DATA

Обработка естественного языка (NLP)

- Давайте посмотрим, как можно применить теорему Байеса для задач классификации данных.
- Мы будем применять эту теорему для задач классификации текстовых данных (natural language processing – обработка естественного языка).

Обработка естественного языка (NLP)

- Для начала применим теорему Байеса для построения модели машинного обучения!

Обработка естественного языка (NLP)

- Мы моделируем вероятность принадлежности к классу при условии, что признаки принимают определённые значения.

$$\mathbf{x} = (x_1, \ldots, x_n)$$

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)} \quad \Rightarrow \quad p(C_k \mid \mathbf{x}) = \frac{p(C_k)\, p(\mathbf{x} \mid C_k)}{p(\mathbf{x})}$$

Обработка естественного языка (NLP)

- Числитель равен совместной вероятности:

$$p(C_k \mid \mathbf{x}) = \frac{p(C_k)\, p(\mathbf{x} \mid C_k)}{p(\mathbf{x})} \Rightarrow p(C_k \mid \mathbf{x}) = \frac{p(C_k, x_1, \ldots, x_n)}{p(\mathbf{x})}$$

Обработка естественного языка (NLP)

- Правило произведения вероятностей позволяет переписать числитель как набор условных вероятностей.

$$
\begin{aligned}
p(C_k, x_1, \ldots, x_n) &= p(x_1, \ldots, x_n, C_k) \\
&= p(x_1 \mid x_2, \ldots, x_n, C_k)\, p(x_2, \ldots, x_n, C_k) \\
&= p(x_1 \mid x_2, \ldots, x_n, C_k)\, p(x_2 \mid x_3, \ldots, x_n, C_k)\, p(x_3, \ldots, x_n, C_k) \\
&= \cdots \\
&= p(x_1 \mid x_2, \ldots, x_n, C_k)\, p(x_2 \mid x_3, \ldots, x_n, C_k) \cdots p(x_{n-1} \mid x_n, C_k)\, p(x_n \mid C_k)\, p(C_k)
\end{aligned}
$$

Обработка естественного языка (NLP)

- Далее мы сделаем предположение, что все признаки X являются **независимыми** друг от друга.
- Это позволяет переписать формулу вот так:

$$p(x_i \mid x_{i+1}, \ldots, x_n, C_k) = p(x_i \mid C_k)$$

Обработка естественного языка (NLP)

- Тогда формулу для модели Наивного Байеса можно переписать следующим образом:

$$
\begin{aligned}
p(C_k \mid x_1, \ldots, x_n) &\propto p(C_k, x_1, \ldots, x_n) \\
&\propto p(C_k)\, p(x_1 \mid C_k)\, p(x_2 \mid C_k)\, p(x_3 \mid C_k)\, \cdots \\
&\propto p(C_k) \prod_{i=1}^{n} p(x_i \mid C_k)\,,
\end{aligned}
$$

Обработка естественного языка (NLP)

- Символ ∝ означает “пропорционально”

$$
\begin{aligned}
p(C_k \mid x_1, \ldots, x_n) &\propto p(C_k, x_1, \ldots, x_n) \\
&\propto p(C_k)\, p(x_1 \mid C_k)\, p(x_2 \mid C_k)\, p(x_3 \mid C_k) \cdots \\
&\propto p(C_k) \prod_{i=1}^{n} p(x_i \mid C_k)\,,
\end{aligned}
$$

Обработка естественного языка (NLP)

- Рассмотрим пример применения получившейся модели Наивного Байеса.

$$
\begin{aligned}
p(C_k \mid x_1, \ldots, x_n) &\propto p(C_k, x_1, \ldots, x_n) \\
&\propto p(C_k)\, p(x_1 \mid C_k)\, p(x_2 \mid C_k)\, p(x_3 \mid C_k) \cdots \\
&\propto p(C_k) \prod_{i=1}^{n} p(x_i \mid C_k)\,,
\end{aligned}
$$

Обработка естественного языка (NLP)

- Существует множество вариаций модели Наивного Байеса, в том числе:
  - Мультиномиальная модель
  - Модель Гаусса
  - Комплементарная модель
  - Модель Бернулли
  - Категориальная модель
  - **Почитайте детали в онлайн-документации!**

Обработка естественного языка (NLP)

- Мы будем работать с мультиномиальной моделью, потому что она наиболее часто встречается в задачах обработки текста.
- Итак – представим себе, что мы хотим создать веб-сайт с отзывами о фильмах. Нам нужно будет разбивать все отзывы на положительные и отрицательные (негативные).

Обработка естественного языка (NLP)

- Предыдущие отзывы были размечены вручную на положительные и отрицательные (негативные).
- В будущем мы с помощью алгоритма машинного обучения будем автоматически классифицировать новые отзывы.

Обработка естественного языка (NLP)

- Но как мы можем обучить модель на текстовых данных?
- Мультиномиальная модель Наивного Байеса достаточно хорошо работает на векторизации слов, когда мы подсчитываем, сколько раз то или иное слово встречается в документе (count vectorization).

Обработка естественного языка (NLP)

- Для начала разделяем два класса:

Обработка естественного языка (NLP)

- Вычисляем вероятности для каждого класса:

Обработка естественного языка (NLP)

- Вычисляем вероятности для каждого класса:

P(pos) = 25/35

P(neg) = 10/35

Обработка естественного языка (NLP)

- Эти вероятности пригодятся нам позже!

P(pos) = 25/35

P(neg) = 10/35

Обработка естественного языка (NLP)

- Далее выполняем подсчёт отдельных слов:

Обработка естественного языка (NLP)

- Далее выполняем подсчёт отдельных слов:

10	2	8	4
movie	actor	great	film

8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- Вычисляем условные вероятности для класса и слова.

10	2	8	4
movie	actor	great	film

8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- Вычисляем условные вероятности:

10	2	8	4
movie	actor	great	film

P(movie|pos)

8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- Вычисляем условные вероятности:

10	2	8	4
movie	actor	great	film

P(movie|pos) = 10/24=0.42

8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- Вычисляем условные вероятности:

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

8	10	0	2
movie	actor	great	film

P(movie|pos) = 10/24 = 0.42
P(actor|pos) = 2/24 = 0.08
P(great|pos) = 8/24 = 0.33
P(film|pos) = 4/24 = 0.17

Обработка естественного языка (NLP)

- Вычисляем условные вероятности:

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

8	10	0	2
movie	actor	great	film

P(movie|neg)= 8/20 = 0.4
P(actor|neg) = 10/20 = 0.5
P(great|neg) = 0/20 = 0
P(film|neg) = 2/20 = 0.1

Обработка естественного языка (NLP)

- Вычисляем условные вероятности:

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

P(movie|neg)= 8/20 = 0.4
P(actor|neg) = 10/20 = 0.5
P(great|neg) = 0/20 = 0
P(film|neg) = 2/20 = 0.1

Обработка естественного языка (NLP)

- А теперь возьмём новый отзыв:

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

“movie actor”

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- А теперь возьмём новый отзыв:

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

“movie actor”

P(pos)

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- Начинаем с вероятности класса

25

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

10

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

“movie actor”

P(pos)

Обработка естественного языка (NLP)

- Начинаем с вероятности класса

25

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

10

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

“movie actor”

P(pos) = (25/35)

Обработка естественного языка (NLP)

- Далее условные вероятности

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

“movie actor”

P(pos)×P(movie|pos)

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- Далее условные вероятности

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

“movie actor”

P(pos)×P(movie|pos)×P(actor|pos)

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- Далее условные вероятности

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

“movie actor”

(0.71)×(0.42)×(0.08)

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- Далее условные вероятности

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

“movie actor”

(0.71)×(0.42)×(0.08) = 0.024

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- Получаем значение 0.024

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

“movie actor”

(0.71)×(0.42)×(0.08) = 0.024

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- Это значение пропорционально P(pos|“movie actor”)

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

“movie actor”

(0.71)×(0.42)×(0.08) = 0.024

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- Это значение пропорционально P(pos|“movie actor”)

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

“movie actor”

0.024 ∝ P(pos| “movie actor”)

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- Далее делаем то же самое для класса “negative”

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

“movie actor”

P(neg)×P(movie|neg)×P(actor|neg)

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- Далее делаем то же самое для класса “negative”

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

“movie actor”

(10/35)×(0.4)×(0.5) = 0.057

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- Это значение пропорционально P(neg|“movie actor”)

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

“movie actor”

0.057 ∝ P(neg| “movie actor”)

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- Сравниваем результаты

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

“movie actor”

0.057 ∝ P(neg| “movie actor”)
0.024 ∝ P(pos| “movie actor”)

Обработка естественного языка (NLP)

- Выбираем наибольшее значение

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

“movie actor”

0.057 ∝ P(neg| “movie actor”)
0.024 ∝ P(pos| “movie actor”)

Обработка естественного языка (NLP)

- В итоге классифицируем отзыв как негативный

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

“movie actor”

0.057 ∝ P(neg| “movie actor”)
0.024 ∝ P(pos| “movie actor”)

Обработка естественного языка (NLP)

- Что делать, если где-то счётчик равен 0 ?

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

“great movie”

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

Обработка естественного языка (NLP)

- Что делать, если где-то счётчик равен 0 ?

0.42	0.08	0.33	0.17
10	2	8	4
movie	actor	great	film

0.4	0.5	0	0.1
8	10	0	2
movie	actor	great	film

“great movie”

Обработка естественного языка (NLP)

- Вероятность равна нулю! Вне зависимости от текста!

25

0.42	**0.08**	**0.33**	**0.17**
10	2	8	4
movie	actor	great	film

“great movie”

P(neg)×P(great|neg)×P(movie|neg)

10

0.4	**0.5**	**0**	**0.1**
8	10	0	2
movie	actor	great	film

PIERIAN DATA

Обработка естественного языка (NLP)

- Параметр Альфа для сглаживания счётчиков:

25

10**+1**	2**+1**	8**+1**	4**+1**
movie	actor	great	film

“great movie”

P(neg)×P(great|neg)×P(movie|neg)

10

8**+1**	10**+1**	0**+1**	2**+1**
movie	actor	great	film

Обработка естественного языка (NLP)

- Вычисляем заново условные вероятности...

25

10**+1**	2**+1**	8**+1**	4**+1**
movie	actor	great	film

10

8**+1**	10**+1**	0**+1**	2**+1**
movie	actor	great	film

Обработка естественного языка (NLP)

- Большие значения альфа будут “сглаживать” слова, уменьшая разницу между отдельными словами.
- Далее мы посмотрим на извлечение признаков из текста в общем случае.
- Есть ли какие-то более продвинутые способы, чем просто подсчёт количества слов?

Извлечение признаков из текстовых данных

Теория и интуиция

PIERIAN DATA

Обработка естественного языка (NLP)

- Большинство классических алгоритмов машинного обучения не могут принимать на вход сырые текстовые данные.
- Нам следует сначала выполнить “извлечение” признаков из текста, чтобы передать уже числовые признаки в алгоритм машинного обучения.

Обработка естественного языка (NLP)

- Основные методы извлечения признаков:
  - Векторизация счётчиков слов (Count Vectorization)
  - TF-IDF:
    - Term Frequency - Inverse Document Frequency

Обработка естественного языка (NLP)

- Векторизация счётчиков слов (Count Vectorization)

You are good
I feel good
I am good

Обработка естественного языка (NLP)

- Создаём словарь всех встречающихся слов

You are good
I feel good
I am good

Обработка естественного языка (NLP)

- Создаём словарь всех встречающихся слов

YOU	ARE	GOOD	I	FEEL	AM

Обработка естественного языка (NLP)

- Создаём вектора со счётчиками

	YOU	ARE	GOOD	I	FEEL	AM
You are good						
I feel good						
I am good						

Обработка естественного языка (NLP)

- Создаём вектора со счётчиками

	YOU	ARE	GOOD	I	FEEL	AM
You are good	1	1	1	0	0	0
I feel good	0	0	1	1	1	0
I am good	0	0	1	1	0	1

Обработка естественного языка (NLP)

- Большинство значений будут нулевыми!

	YOU	ARE	GOOD	I	FEEL	AM
You are good	1	1	1	0	0	0
I feel good	0	0	1	1	1	0
I am good	0	0	1	1	0	1

Обработка естественного языка (NLP)

- Векторизация счётчиков слов (Count Vectorization)

```
messages = ["Hey, lets go to the game today!",
            "Call your sister.",
            "Want to go walk your dogs?"]
```

Обработка естественного языка (NLP)

- Document Term Matrix (DTM)

call	dogs	game	go	hey	lets	sister	the	to	today	walk	want	your
0	0	1	1	1	1	0	1	1	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0	0	1
0	1	0	1	0	0	0	0	1	0	1	1	1

Обработка естественного языка (NLP)

- Векторизация счётчиков слов (Count Vectorization) создаёт признак для каждого слова, а счётчик “сколько раз встретилось слово” характеризует “силу” признака/слова.
- Для больших документов матрицы хранятся как **разреженные (sparse)** для экономии ресурсов компьютера.

Обработка естественного языка (NLP)

- На что обратить внимание:
  - Частые слова (такие как “a” , “the”, “about”).
  - Слова, частые в целом наборе документов (например, слово “run” в наборе спортивных статей).

Обработка естественного языка (NLP)

- Стоп-слова – это слова, встречающиеся настолько часто, что их можно безопасно удалить из рассмотрения.
- Большинство библиотек NLP имеют встроенные списки стоп-слов.

Обработка естественного языка (NLP)

- Чтобы учесть частоту слов во всех документах, можно применить процесс векторизации TF-IDF.
- В этом случае в матрицу DTM пишется не просто частота слов, а для каждого слова вычисляется значение “term frequency-inverse document frequency” (TF-IDF).

Обработка естественного языка (NLP)

- Term frequency tf(t,d): сколько раз слово t встречается в документе d.
- Однако, метрика “Term Frequency” сама по себе недостаточна для анализа текста!
- Представим себе часто встречающиеся слова – такие как “a” или “the”...

Обработка естественного языка (NLP)

- Поскольку слово "the" встречается очень часто, метрика “term frequency” будет уделять больше внимания тем документам, где слово "the" встречается чаще. Вместо того, чтобы давать больше веса таким словам, как "red" или "dogs".

Обработка естественного языка (NLP)

- Также нужно учитывать ситуации, когда какие-то слова встречаются во всём наборе документов, но они не являются стоп-словами:
  - Например, слово “run” в документах о спорте.

Обработка естественного языка (NLP)

- Множитель “inverse document frequency” позволяет уменьшить вес слов, которые встречаются часто в наборе документов, а также увеличить вес тех слов, которые встречаются редко.

Обработка естественного языка (NLP)

- Вычисляется IDF так: берётся общее количество документов, делится на количество документов, в которых содержится рассматриваемое слово, и затем берётся логарифм этой дроби.

Обработка естественного языка (NLP)

- Множитель IDF говорит о том, как часто или редко встречается слово во всём наборе документов.
- Чем ближе IDF к 0, тем чаще встречается слово.
- Вычисляется IDF так: берётся общее количество документов, делится на количество документов, в которых содержится рассматриваемое слово, и затем берётся логарифм этой дроби.
- .

Обработка естественного языка (NLP)

- **TF-IDF = term frequency * (1 / document frequency)**
- **TF-IDF = term frequency * inverse document freq**

$$\mathrm{tfidf}(t, d, D) = \mathrm{tf}(t, d) \cdot \mathrm{idf}(t, D)$$

$$\mathrm{idf}(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|}$$

Обработка естественного языка (NLP)

- К счастью, Scikit-learn может вычислять все эти метрики с помощью готовых методов.
- Синтаксис похож на работу с другими моделями в Scikit-Learn!

Обработка естественного языка (NLP)

```
from sklearn.feature_extraction.text import TfidfVectorizer

vect = TfidfVectorizer()
dtm = vect.fit_transform(messages)
```

call	dogs	game	go	hey	lets	sister	the	to	today	walk	want	your
0.000	0.00	0.403	0.307	0.403	0.403	0.000	0.403	0.307	0.403	0.00	0.00	0.000
0.623	0.00	0.000	0.000	0.000	0.000	0.623	0.000	0.000	0.000	0.00	0.00	0.474
0.000	0.46	0.000	0.349	0.000	0.000	0.000	0.000	0.349	0.000	0.46	0.46	0.349

Обработка естественного языка (NLP)

- TF-IDF позволяет понять слова в контексте всего набора слов во всех документах, а не только в одном документе.
- Далее мы посмотрим, как выполняются эти операции в Python и SciKit-Learn!

Извлечение признаков из текстовых данных

Подсчитываем слова вручную

PIERIAN DATA

Обработка естественного языка (NLP)

- Для начала мы вручную построим модель “мешок слов” (“bag of words”).
- Это будет подсчёт того, как часто то или иное слово встречается в документе (count vectorization).
- Давайте начнём!

Извлечение признаков из текстовых данных

Работа с Scikit-Learn

PIERIAN DATA

Классификация текстовых данных

Часть 1: Анализ данных и признаки

PIERIAN DATA

Классификация текстовых данных

Часть 2: Построение моделей

PIERIAN DATA

Проверочные задания Классификация текста

Обзор заданий

PIERIAN DATA

Проверочные задания
Классификация текста

Решения

PIERIAN DATA